Solution: Remove All Adjacent Duplicates In String

Problem Statement

Give a string `s` and convert it into a valid string.

A string is considered valid if it does not have any two adjacent duplicate characters. The task is to remove the minimum number of characters from `s` such that it becomes valid. The removal of characters should be done in a way that no further removals are required.

Examples

Example 1

- Input: `"abbaca"`
- Expected Output: `"ca"`
- Description: We remove 'b' from "abbaca" to get "aaca", then remove 'a' from "aaca" to get "ca"

Example 2

- Input: `"azxxzy"`
- Expected Output: `"ay"`
- Description: We remove 'x' from "azxxzy" to get "azzy", then remove 'z' from "azzy" to get "ay"

Example 3

- Input: `"abba"`
- Expected Output: `""`
- Description: We remove 'b' from "abba" to get "aa", then remove 'a' from "aa" to get an empty string

Solution

To solve this problem, we can utilize a stack data structure. The stack helps us to keep track of the characters in the string as we iterate through it. For each character in the string, we compare it with the top element of the stack. If they match, it indicates a pair of adjacent duplicates, and we remove the top element from the stack. Otherwise, we add the current character to the stack. This approach effectively removes all adjacent duplicate pairs.

Once we have processed all characters, the remaining elements in the stack represent the string with all adjacent duplicates removed. The final step involves converting the stack contents back into a string in the original order, which can be achieved by reversing the stack.

Detailed Step-by-Step Walkthrough

1. Initialize an empty stack
2. Iterate over each character in the input string
3. For each character, check if the stack is not empty and the top of the stack is equal to the current character
    - If it is, pop the top element from the stack
    - If it's not, push the current character into the stack
4. After the iteration, convert the stack into a string. The order of the characters in the stack is the reverse of the final valid string, so reverse the stack's string before returning
5. Return the final valid string.

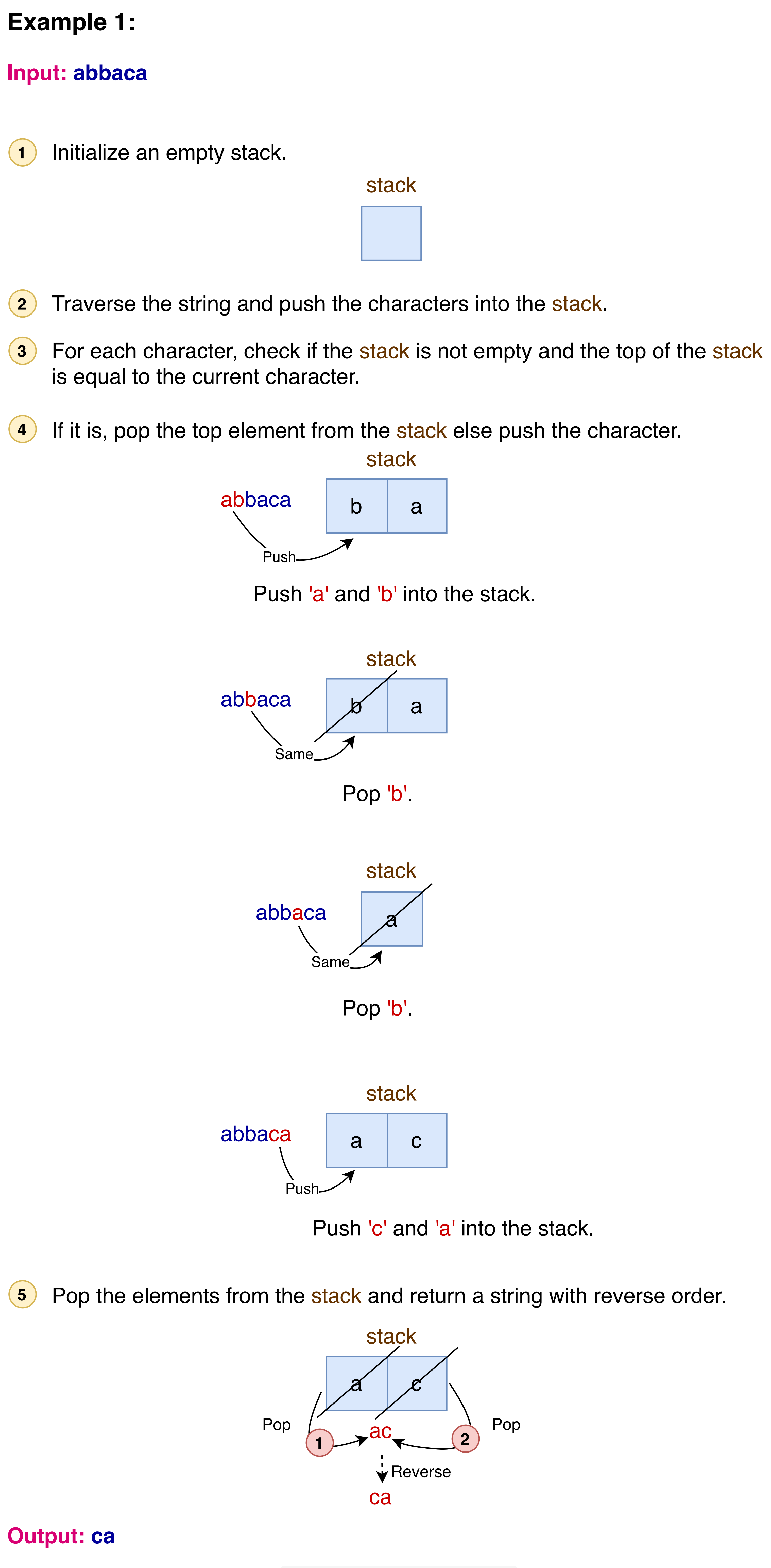
Algorithm Walkthrough

Let's walk through the example "abbaca" step by step using the stack-based algorithm:

1. **Initialize an Empty Stack**: Start with an empty stack to keep track of the characters.
2. **Process First Character** ('**a**'):
    - Current Character: `'a'`
    - Since the stack is empty, push 'a' onto the stack.
    - Stack after operation: `['a']`
3. **Process Second Character** ('**b**'):
    - Current Character: `'b'`
    - 'b' is not the same as the top element of the stack ('a'), so push 'b'.
    - Stack after operation: `['a', 'b']`
4. **Process Third Character** ('**b**'):
    - Current Character: `'b'`
    - 'b' matches the top element of the stack, so pop the top element ('b').
    - Stack after operation: `['a']`
5. **Process Fourth Character** ('**a**'):
    - Current Character: `'a'`
    - 'a' matches the top element of the stack, so pop the top element ('a').
    - Stack after operation: `[]`
6. **Process Fifth Character** ('**c**'):
    - Current Character: `'c'`
    - Stack is empty, so push 'c'.
    - Stack after operation: `['c']`
7. **Process Sixth Character** ('**a**'):
    - Current Character: `'a'`
    - 'a' is not the same as the top element of the stack ('c'), so push 'a'.
    - Stack after operation: `['c', 'a']`
8. **Final Step - Convert Stack to String**:
    - The final stack is `['c', 'a']`.
    - Convert this stack to a string by concatenating the elements in order.
    - The resulting string is `"ca"`.

So, the output for the input string "abbaca" is "ca", as all adjacent duplicates have been successfully removed.

Here is the visual representation of the algorithm:



Example 1:

Input: abbaca

① Initialize an empty stack.

② Traverse the string and push the characters into the stack.

③ For each character, check if the stack is not empty and the top of the stack is equal to the current character.

④ If it is, pop the top element from the stack else push the character.

Push 'a' and 'b' into the stack.

Pop 'b'.

Pop 'b'.

Push 'c' and 'a' into the stack.

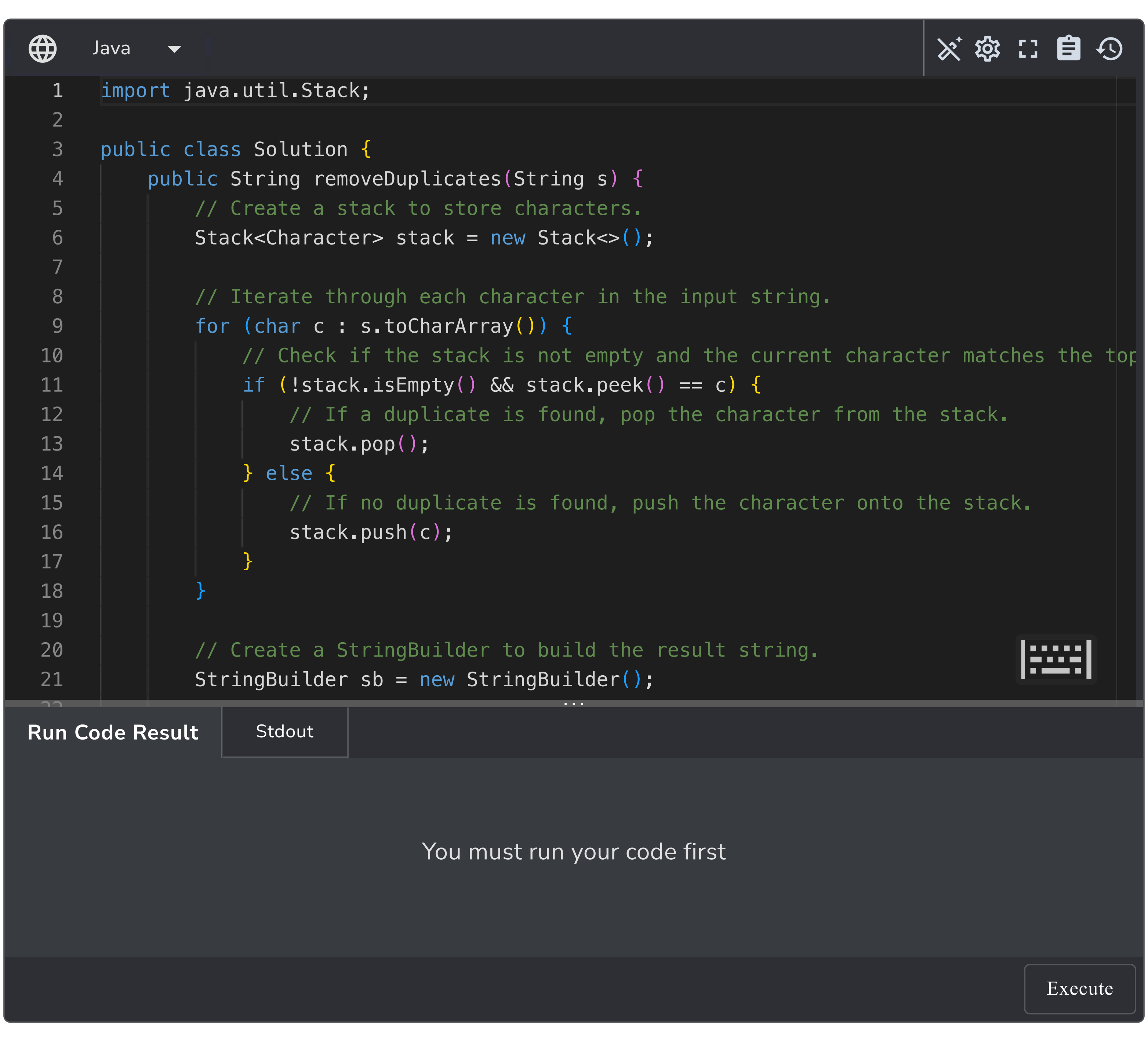
⑤ Pop the elements from the stack and return a string with reverse order.

Output: ca

Remove All Adjacent Duplicates

Code

Here is the code for this algorithm:



```java
import java.util.Stack;

public class Solution {
    public String removeDuplicates(String s) {
        // Create a stack to store characters.
        Stack<Character> stack = new Stack<>();

        // Iterate through each character in the input string.
        for (char c : s.toCharArray()) {
            // Check if the stack is not empty and the current character matches the top
            if (!stack.isEmpty() && stack.peek() == c) {
                // If a duplicate is found, pop the character from the stack.
                stack.pop();
            } else {
                // If no duplicate is found, push the character onto the stack.
                stack.push(c);
            }
        }

        // Create a StringBuilder to build the result string.
        StringBuilder sb = new StringBuilder();
```

You must run your code first

Time and Space Complexity

The time complexity of the algorithm is $O(n)$, where n is the length of the string. This is because we perform a single pass over the string.

The space complexity of the algorithm is $O(n)$, where n is the length of the string. In the worst case, if there are no adjacent duplicates, the stack will contain all the characters of the string.

← Previous
Problem 7: Remove All Adjacent Duplicates In String

Next →
Problem 8: Removing Stars From A String

☐ Mark as Completed